

OBJETIVO DE LA CLASE

Aprenderás:

1. Qué son **Flexbox** y **Grid**
2. Cómo funcionan sus **conceptos fundamentales**
3. Cuáles son sus **propiedades principales**
4. Cuándo usar uno u otro (Flexbox vs Grid)

1. ¿Qué es Flexbox?

Flexbox (Flexible Box Layout) es un **sistema unidimensional de diseño**.

Sirve para organizar elementos **en fila o en columna**, distribuyendo el espacio automáticamente y alineando los ítems con facilidad.

→ Lo usas cuando quieres **alinear cosas dentro de un contenedor**, por ejemplo:

- Menús horizontales
- Tarjetas alineadas
- Elementos centrados vertical y horizontalmente

◆ Estructura básica

```
.contenedor {  
  display: flex;  
}
```

Todo elemento dentro de .contenedor se convierte en un **flex item**.

2. Propiedades principales de Flexbox

Se dividen en **propiedades del contenedor** y **de los ítems**.

A. Propiedades del contenedor

display: flex;

Activa Flexbox.

flex-direction

Define la **dirección principal del eje** (main axis):

Valor	Dirección
row	Horizontal (por defecto)
row-reverse	Horizontal inversa
column	Vertical
column-reverse	Vertical inversa

Ejemplo:

```
.contenedor{  
  display: flex;  
  flex-direction: row; /* o column */  
}
```

justify-content

Alinea los ítems **en el eje principal**:

Valor	Efecto
flex-start	Al inicio
flex-end	Al final
center	Centrado
space-between	Espaciados con hueco entre ellos
space-around	Espaciados con margen a los lados
space-evenly	Espaciados iguales entre todos

align-items

Alinea los ítems **en el eje transversal (secundario)**:

Valor	Efecto
stretch	Estira (por defecto)

Valor	Efecto
-------	--------

flex-start	Arriba
------------	--------

flex-end	Abajo
----------	-------

center	Centrado verticalmente
--------	------------------------

baseline	Alinea según línea de texto
----------	-----------------------------

flex-wrap

Permite que los ítems **salten de línea** si no caben:

Valor	Efecto
-------	--------

nowrap	Todo en una sola línea
--------	------------------------

wrap	Salta a la siguiente línea
------	----------------------------

wrap-reverse	Salta en sentido inverso
--------------	--------------------------

align-content

Alinea **las filas** (cuando hay varias por el wrap):

Valor	Efecto
-------	--------

flex-start	Arriba
------------	--------

flex-end	Abajo
----------	-------

center	Centrado
--------	----------

space-between	Espacio entre filas
---------------	---------------------

space-around	Espacios iguales alrededor
--------------	----------------------------

stretch	Estira las filas
---------	------------------

B. Propiedades de los ítems (hijos)

flex-grow

Define cuánto **crece** un ítem respecto a los demás.

```
.item1 { flex-grow: 2; } /* crece el doble que los otros */
```

```
.item2 { flex-grow: 1; }
```

flex-shrink

Define cuánto **se reduce** un ítem cuando falta espacio.

flex-basis

Define el **tamaño inicial** del ítem (antes de crecer o encogerse).

flex

Atajo que combina las tres:

```
.item {  
  flex: 1 0 200px; /* grow | shrink | basis */  
}
```

align-self

Permite que un ítem **anule la alineación general** del contenedor:

```
.item {  
  align-self: flex-end;  
}
```

Ejemplo práctico de Flexbox

```
<div class="contenedor">
```

```
  <div class="item">1</div>
```

```
  <div class="item">2</div>
```

```
  <div class="item">3</div>
```

```
</div>
```

```
.contenedor {
```

```
  display: flex;
```

```
  justify-content: space-around;
```

```
  align-items: center;
```

```
  height: 200px;
```

```
background: #eee;
}
```

```
.item {
background: #0077ff;
color: white;
padding: 1rem;
border-radius: 8px;
}
```

3. ¿Qué es CSS Grid?

Grid Layout es un **sistema bidimensional de diseño**.

Te permite controlar **filas y columnas** simultáneamente.

Ideal para **estructuras completas de página** (layouts, galerías, dashboards...).

◆ Estructura básica

```
.contenedor {
display: grid;
}
```

Cada hijo del contenedor es un **grid item**.

4. Propiedades principales de Grid

A. Propiedades del contenedor

grid-template-columns y grid-template-rows

Definen cuántas columnas y filas habrá, y sus tamaños:

```
.contenedor {
display: grid;

grid-template-columns: 200px 1fr 2fr;

grid-template-rows: 100px auto;
```

```
}
```

- px: tamaño fijo
- fr: fracción del espacio disponible
- auto: se ajusta al contenido
- %: porcentaje

gap

Espaciado entre filas y columnas (puede ser row-gap y column-gap).

```
.contenedor {  
  gap: 20px;  
}
```

justify-items / align-items

Alinean los ítems **dentro de sus celdas**:

Valor	Efecto
-------	--------

start	Al inicio
-------	-----------

center	Centrado
--------	----------

end	Al final
-----	----------

stretch	Estira (por defecto)
---------	----------------------

justify-content / align-content

Alinean **toda la cuadrícula dentro del contenedor**.

grid-template-areas

Permite nombrar zonas de la cuadrícula para maquetar más fácilmente:

```
.contenedor {  
  display: grid;
```

```
grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
grid-template-columns: 200px 1fr;  
}
```

Y en HTML:

```
<div class="header"></div>  
<div class="sidebar"></div>  
<div class="main"></div>  
<div class="footer"></div>
```

Con CSS:

```
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main { grid-area: main; }  
.footer { grid-area: footer; }
```

B. Propiedades de los ítems

grid-column y grid-row

Permiten que un ítem ocupe varias celdas.

```
.item1 {  
    grid-column: 1 / 3; /* de la columna 1 a la 3 */  
    grid-row: 1 / 2;  
}
```

justify-self / align-self

Alinean **un solo ítem** dentro de su celda (igual que en Flexbox).

Ejemplo práctico de Grid

```

<div class="grid">

  <div class="item a">A</div>

  <div class="item b">B</div>

  <div class="item c">C</div>

  <div class="item d">D</div>

</div>

.grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 10px;
  background: #eee;
  padding: 10px;
}

.item {
  background: #3b82f6;
  color: white;
  padding: 1rem;
  text-align: center;
  border-radius: 8px;
}

```

5. Flexbox vs Grid — ¿Cuándo usar cada uno?

Situación	Usa Flexbox	Usa Grid
Alinear elementos en una sola fila o columna	✓	
Crear un diseño de página completo (con filas y columnas)		✓
Distribuir espacio dinámico entre ítems	✓	✓

Situación	Usa Flexbox	Usa Grid
Ordenar elementos de manera visual sin alterar el HTML		✓
Centrar elementos fácilmente	✓	✓
Galerías, dashboards, layouts complejos		✓

→ **Regla general moderna:**

- Usa **Grid** para la estructura general.
- Usa **Flexbox** dentro de las secciones (por ejemplo, en el menú o en una tarjeta).

6. Propiedades modernas útiles

- `place-items: center;` → combina `align-items` y `justify-items`
- `place-content: center;`
- `minmax()` → define un rango flexible:
- `grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));`
- `auto-fit` y `auto-fill` → adaptan automáticamente el número de columnas según el ancho disponible.

7. Ejemplo combinado de Flexbox + Grid

```
header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```
main {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 2rem;
```

}

8. Recomendaciones para un diseño moderno

- Usa **Grid** para layout principal y **Flexbox** dentro de componentes.
- Combina con **media queries** para hacerlo **responsive**.
- Usa **variables CSS** para colores y espaciados.
- Aprovecha gap (ya es compatible con Flexbox también).
- Usa herramientas visuales como [CSS Grid Generator](#) o Flexbox Froggy para practicar.